

Laboratory Assignment 8: High-Performance Big Data Analytics Using R

1. Problem Statement

The New York City Taxi and Limousine Commission (NYC TLC) generates millions of taxi-trip records containing information such as pickup and drop-off locations, trip timestamps, passenger count, trip distance, fare amount, payment type, and total transaction amount. Processing and analyzing such large-scale datasets using conventional approaches can lead to increased execution time and memory consumption.

Develop a **high-performance and scalable data analytics solution in R** using the NYC Taxi Trip dataset. The objective is to analyze large-scale transportation data while investigating different R programming approaches for improving computational efficiency.

Use **data.table** for efficient data manipulation, filtering, joining, grouping, and aggregation. Apply functional programming techniques such as **apply()**, **lapply()**, and/or **purrr::map()** for repeated analytical operations and compare them with suitable **vectorized and data.table implementations**.

Further, implement selected computationally intensive operations using both **sequential and parallel processing** with **foreach** and **doParallel**. Benchmark different implementations using **microbenchmark**, **system.time()**, or equivalent profiling techniques.

The analysis should identify important taxi travel patterns, high-demand periods, high-revenue routes, fare and trip-distance characteristics, and payment preferences. Based on experimental results, critically evaluate the different R programming approaches in terms of **execution time, memory efficiency, scalability, readability, and computational performance**.

2. Dataset

Dataset Name: NYC Taxi & Limousine Commission (TLC) – Yellow Taxi Trip Records

Official

Dataset

Link:

<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

Students should use the **Yellow Taxi Trip Records** available on the official NYC TLC website.

The dataset contains millions of trip records with attributes such as:

- Pickup date and time
- Drop-off date and time
- Pickup Location ID
- Drop-off Location ID
- Passenger count
- Trip distance
- Payment type
- Fare amount
- Tip amount
- Tolls amount
- Total amount

Students should also use the **Taxi Zone Lookup Table** available on the same website to map pickup and drop-off Location IDs to their corresponding **Zone and Borough**.

A sufficiently large subset of the data should be selected so that the performance differences between conventional, optimized, sequential, and parallel implementations can be meaningfully demonstrated.

3. Tasks to be Performed

Task 1: Data Acquisition and Preparation

1. Import the selected NYC Yellow Taxi Trip dataset into R.
 2. Inspect the structure, dimensions, data types, and summary statistics of the dataset.
 3. Identify and handle missing, duplicate, inconsistent, or invalid records.
 4. Convert attributes into appropriate data types.
 5. Extract relevant temporal attributes such as:
 - Hour
 - Day
 - Day of week
 - Month
 6. Remove unnecessary attributes where appropriate to reduce memory utilization.
 7. Import the Taxi Zone Lookup Table.
 8. Join taxi-trip records with zone information using pickup and drop-off Location IDs.
 9. Use `data.table` for efficient filtering, joining, grouping, and aggregation.
-

4. Task 2: Transportation Data Analytics

Perform suitable analytical operations to:

1. Calculate the **number of taxi trips by hour of day**.
2. Compare taxi demand across different **days of the week**.
3. Analyze monthly variations in taxi usage.
4. Calculate **average fare and total revenue** for different time periods.
5. Identify the **most frequently travelled pickup-drop-off routes**.
6. Identify the **highest-revenue routes** based on total amount collected.
7. Analyze the relationship between **trip distance and fare amount**.
8. Compare different **payment methods across boroughs or locations**.
9. Identify any additional meaningful or unusual patterns from the dataset.

Students should provide appropriate interpretations for the analytical results obtained.

5. Task 3: Functional and Efficient R Programming

Implement selected analytical operations using functional programming techniques such as:

- `apply()`
- `lapply()`
- `purrr::map()`

Implement the same or equivalent operations using:

- Vectorized R operations
- `data.table`

Compare the different implementations based on:

- Execution time
- Computational efficiency
- Code complexity
- Readability
- Memory utilization
- Suitability for large-scale datasets

Students should explain the reasons for observed performance differences.

6. Task 4: Parallel Computing

Divide the dataset into suitable logical partitions, such as **month-wise subsets**.

Perform a selected computationally intensive analytical operation using:

A. Sequential Processing

Execute the selected analysis sequentially and record its execution time.

B. Parallel Processing

Implement the same operation using:

- `foreach`
- `doParallel`

Configure an appropriate number of processing cores and execute the analysis in parallel.

Compare sequential and parallel execution times.

Calculate the speedup as:

Speedup = Sequential Execution Time / Parallel Execution Time

Discuss whether parallel processing provides a significant performance improvement and identify possible reasons for the observed result.

7. Task 5: Performance Benchmarking and Optimization

Use **microbenchmark**, **system.time()**, or another suitable profiling technique to compare selected R implementations.

Perform appropriate comparisons such as:

- Base R vs. data.table
- Functional programming vs. vectorized programming
- Functional programming vs. data.table
- Sequential vs. parallel processing

Prepare a comparison table containing the execution times obtained using different approaches.

Analyze:

- Execution time
- Memory requirements
- Scalability
- Programming complexity
- Readability

Explain why certain approaches perform better for large-scale datasets.

8. Task 6: Data Visualization

Use **ggplot2** to create appropriate visualizations representing important findings.

Suitable visualizations may include:

- Number of trips by hour
- Taxi demand by day of week
- Monthly taxi demand
- Average fare trends
- Revenue trends
- Top pickup locations
- Top drop-off locations
- Most frequently travelled routes
- Highest-revenue routes
- Payment-type distribution
- Trip distance vs. fare amount

All visualizations should contain appropriate:

- Titles

- Axis labels
- Legends
- Units
- Brief interpretation

Optional Advanced Task: Use **leaflet** to create an interactive geographical visualization representing important taxi zones or high-value routes.

9. Task 7: Critical Analysis and Recommendation

Based on the experiments performed, critically evaluate the different R programming approaches.

Your analysis should address:

1. Which approach provides the best execution performance?
2. How does `data.table` improve the processing of large datasets?
3. When is functional programming preferable?
4. When do vectorized operations provide better performance?
5. What performance improvement was obtained using parallel computing?
6. Does parallel computing always result in faster execution? Justify using your experimental results.
7. What trade-offs exist between execution speed, memory utilization, code readability, and scalability?
8. Which R programming approach would you recommend for large-scale analytical applications?

The final recommendation must be supported by the **experimental and benchmarking results** obtained during the laboratory work.

10. Expected Outcomes

After completing this laboratory assignment, students should be able to:

1. Process and analyze **large-scale real-world datasets using R**.
2. Perform efficient data manipulation, filtering, grouping, joining, and aggregation using **`data.table`**.
3. Apply functional programming techniques using **`apply()`, `lapply()`, and `purrr::map()`**.
4. Implement and evaluate **vectorized R operations**.
5. Develop parallel processing solutions using **`foreach` and `doParallel`**.
6. Compare sequential and parallel implementations using performance measurements.
7. Use benchmarking techniques to evaluate different R programming approaches.
8. Analyze execution time, memory efficiency, and scalability of alternative implementations.
9. Generate meaningful visualizations using **`ggplot2`** and optionally `leaflet`.

10. Interpret large-scale transportation data and derive meaningful analytical insights.
 11. Critically evaluate alternative implementation strategies.
 12. Design an **efficient and scalable end-to-end data analytics workflow in R**.
-

11. Deliverables

Students are required to submit:

- Complete R implementation
 - Data preprocessing and cleaning steps
 - data.table implementation
 - Functional programming implementation
 - Vectorized implementation
 - Sequential processing implementation
 - Parallel processing implementation
 - Performance benchmark results
 - Execution-time comparison table
 - Relevant data visualizations
 - Interpretation of analytical findings
 - Critical performance analysis
 - Final recommendation and conclusion
-

12. Submission Guidelines

Notebook Submission

Submit the completed **Google Colab/Jupyter Notebook in PDF format**.

The notebook should clearly contain:

Problem Statement → Dataset → Data Preparation → Implementation → Analysis → Benchmarking → Visualizations → Observations → Critical Analysis → Conclusion

Git Submission

Upload the complete implementation/notebook to your **Git repository**.

The **Git repository link must be included in the final submission**.

The submitted implementation should be properly organized, documented, and reproducible.

13. Suggested R Packages

Students may use the following R packages:

- data.table
- ggplot2
- foreach
- doParallel
- microbenchmark
- purrr
- lubridate

Optional: leaflet

14. AI Feedback Scores

Rating Scale: 1 – Lowest | 4 – Highest

Parameter	Score
Real-life Application	4 / 4
Problem-solving Ability	4 / 4
Critical Reasoning	4 / 4
Creativity	4 / 4
Alignment with Learning / Course Outcomes	4 / 4

Overall Assessment

The assignment demonstrates a high level of real-world applicability by requiring students to process a large-scale transportation dataset consisting of millions of transactional records. Students are required not only to perform data analysis but also to investigate alternative computational approaches and justify the selection of efficient techniques.

The comparative analysis of functional, vectorized, data.table, sequential, and parallel implementations introduces significant **problem-solving and critical-reasoning components**. Students must interpret benchmark results rather than merely execute predefined commands.

The integration of big-data analytics, functional programming, parallel computation, optimization, benchmarking, visualization, and critical evaluation within a single analytical problem also provides substantial scope for creativity and independent analytical decisions.

15. Estimated Time Required

Estimated Total Time: 1 Hour 45 Minutes – 2 Hours

Data Preparation and data.table Analytics – 25–30 Minutes

- Import and preprocess the NYC Taxi dataset.
- Perform datatype conversion and filtering.
- Join trip data with the Taxi Zone Lookup Table.

- Perform efficient grouping and aggregation using `data.table`.

Functional Programming and Code Optimization – 20–25 Minutes

- Implement selected operations using `apply()`, `lapply()`, and/or `purrr::map()`.
- Develop equivalent vectorized or `data.table` implementations.
- Compare the alternative programming approaches.

Parallel Processing and Benchmarking – 25–30 Minutes

- Partition data into suitable subsets.
- Implement sequential processing.
- Implement parallel processing using `foreach` and `doParallel`.
- Benchmark execution time.
- Calculate and interpret speedup.

Visualization and Interpretation – 20–25 Minutes

- Generate analytical visualizations using `ggplot2`.
- Interpret travel, fare, revenue, and payment patterns.
- Summarize the benchmark results.
- Provide final observations and recommendations.

16. COs Linked with the Assignment

CO3, CO4, CO5, CO6

CO3 – Strongly Addressed

Students implement advanced R programming techniques to process, manipulate, analyze, and visualize a large-scale real-world dataset.

CO4 – Strongly Addressed

Students investigate computational efficiency through code optimization, memory-efficient processing, performance benchmarking, vectorization, and parallel computing.

CO5 – Addressed

Students compare multiple analytical and computational approaches and evaluate their suitability based on performance, scalability, readability, and memory requirements.

CO6 – Strongly Addressed

Students design a complete end-to-end analytical workflow involving data acquisition, preprocessing, analytics, optimization, visualization, performance evaluation, and reporting.

17. LOs Linked with the Assignment

LO 5.1, LO 5.2, LO 5.3, LO 6.1, LO 6.2, LO 6.3

The assignment integrates Learning Outcomes associated with:

- Large-scale data processing
 - Efficient data manipulation
 - Functional programming
 - Code optimization
 - Performance benchmarking
 - Parallel computing
 - Memory-efficient programming
 - Analytical visualization
 - Performance evaluation
 - Development of scalable analytical workflows
-

18. Justification

This laboratory assignment provides students with practical exposure to **high-performance data analytics, functional programming, parallel computing, performance benchmarking, and computational optimization** using a realistic large-scale transportation dataset. The NYC Taxi Trip dataset contains millions of transaction records and therefore provides an appropriate environment for demonstrating the practical limitations of conventional data-processing approaches and the benefits of optimized R programming techniques.

Students use **data.table** to perform high-performance data manipulation, filtering, joins, grouping, and memory-efficient aggregations. The requirement to combine taxi-trip information with the Taxi Zone Lookup Table further exposes students to realistic data-integration operations commonly encountered in data engineering and analytics applications.

The assignment also develops proficiency in **functional programming** by requiring students to implement selected analytical operations using `apply()`, `lapply()`, and/or `purrr::map()`. These implementations are compared with vectorized and `data.table` approaches, enabling students to understand that different programming paradigms can produce the same analytical result while differing significantly in execution efficiency, memory utilization, readability, and scalability.

The incorporation of **parallel computing using foreach and doParallel** provides practical exposure to computational optimization. By executing equivalent analyses sequentially and in parallel, measuring their execution times, and calculating speedup, students gain an experimental understanding of the advantages and overhead associated with parallel computation.

Performance profiling using **microbenchmark**, `system.time()`, or equivalent techniques requires students to support their conclusions using measurable experimental evidence. Rather than simply assuming that one programming technique is superior, students must evaluate alternative implementations and critically interpret their benchmark results.

The assignment strongly supports **CO3** by requiring students to implement advanced analytical workflows in R for processing and analyzing large-scale datasets. It directly supports **CO4** through performance benchmarking, code optimization, memory-efficient programming, vectorization, and parallel computation. The comparative evaluation of alternative programming approaches contributes to **CO5**, as students must assess and select suitable computational strategies based on experimental

evidence. The complete workflow—from data acquisition and preprocessing through analytics, optimization, visualization, performance evaluation, and reporting—strongly supports **CO6** by requiring students to design a scalable end-to-end data analytics solution.

The integration of **big-data processing, functional programming, vectorization, parallel computation, benchmarking, visualization, and critical performance evaluation within a single real-world problem** makes the assignment highly relevant to contemporary data science, transportation analytics, and enterprise-scale computing applications.

19. AI Tool Used

ChatGPT

20. Prompt(s) Used

Prompt:

"Evaluate the given R Programming laboratory assignment using a 1–4 rubric for Real-life Application, Problem-solving Ability, Critical Reasoning, Creativity, and Alignment with Learning / Course Outcomes. Identify the estimated time required, map the relevant Course Outcomes (CO1–CO6) and Learning Outcomes (LOs), and justify the ratings based on the R Programming syllabus covering large-scale data analytics, data.table, functional programming using apply(), lapply() and purrr::map(), vectorized programming, parallel computing using foreach and doParallel, performance benchmarking, code optimization, memory-efficient programming, data visualization, and scalable end-to-end data analytics workflows."

21. Academic Responsibility Declaration

I hereby declare that:

1. The assignment/examination has been designed in alignment with the **approved curriculum and intended Learning/Course Outcomes**.
2. The assessment incorporates the following key elements:
 - **Problem-solving ability**
 - **Critical reasoning**
 - **Application of computational concepts to real-world data**
3. An AI tool has been used only for **reviewing and enhancing the assessment quality**, where applicable, and not as a replacement for academic judgment.
4. The assignment has been reviewed, finalized, and approved by me, and I take full academic responsibility for its **content, rigor, fairness, and integrity**.
5. I understand that submission of this declaration is mandatory for all assignments and that it must be presented during the **Meeting with the Chief Quality Officer (CQO) or any quality audit**, wherever applicable.